UNITED INTERNATIONAL UNIVERSITY (UIU)
Dept. of Computer Science & Engineering
Trimester: Summer 2026 (CT-2)
Course No: CSE 3812 **Title:** Artificial Intelligence Laboratory
Section: H **Time:** 45 mins **Marks:** 20

Instructions:

- While answering the questions make sure to follow this convention,
    - Suppose for me, my roll was 1907016. So, the filename should be `1907016_CT2.ipynb`. For you, only the roll number part will change. `[If someone wants to write in a (dot)py script, that's completely fine too]`
    - Properly save the scripts at regular intervals as you'll be coding on an offline basis because unlike google colab, offline codes sometimes don't have autosave option on.
- Then, upload the file in this Link.

- Kindly, **don't use any LLMs to generate your codes**. I will instantly understand whether you wrote it or any LLM wrote it. Any use of LLMs will result in instant disqualification.
- Boilerplate Code

Intelligent University Exam Timetabling

At United International University, Lecturer icé is attempting to automate the final examination schedule for the Department of Computer Science and Engineering. Historically, manual scheduling has resulted in disastrous logistical conflicts, faculty overexertion, and student dissatisfaction. You are tasked with helping him to upgrade a basic, incomplete Constraint Satisfaction Problem (CSP) solver into an Intelligent Timetabling Solver.

You are provided with a Python boilerplate file. This boilerplate currently contains a naive solver that utilizes chronological backtracking and only checks for basic room overlaps. Your objective is to fundamentally rewrite the constraint validation logic and the search heuristics to satisfy a complex set of administrative rules.

Entities and Resources

1. **Courses (Variables):** A dynamic array of computer science courses requiring scheduled examinations.

2. **Time Slots:** The examination period spans multiple days, featuring two distinct sessions per day: Morning (AM) and Afternoon (PM). Time slots are represented as multidimensional tuples: (Day_Number, Slot_Type).

3. **Rooms:** A list of available venues. Venues are classified by type. Standard rooms support **theoretical exams**, while specialized Computer Labs are reserved strictly for **practical, machine-based examinations**.

4. **Domains:** The domain for any given course variable is the Cartesian product of all available time slots and all available rooms. Assigning a value to a course means mapping it to a specific ((Day, Session), Room) tuple.

Course Metadata

Each course possesses specific metadata dictating its logistical requirements:

- batch: The specific student cohort enrolled in the course.
- invigilator: The faculty member assigned to proctor the examination.
- lab_required: A Boolean value (True/False) indicating if the exam requires a Computer Lab venue.

Strict Hard Constraints

The is_consistent function in your boilerplate currently only checks if a room is double-booked. You must completely rewrite this function to validate all of the following conditions before finalizing any assignment:

1. **Specialized Room Constraint (Unary):** If a course's metadata indicates lab_required == True, it strictly must be scheduled in a room containing the string 'Lab'. Conversely, to preserve scarce laboratory resources, if a course does not require a lab (lab_required == False), it strictly must *not* be scheduled in a Lab venue.

2. **Room Overlap Constraint (Binary):** No two distinct courses can be scheduled in the same room during the exact same time slot.

3. **Student Batch Conflict (Binary):** A specific student cohort (batch) cannot be scheduled to sit for two different examinations during the exact same time slot.

4. **Faculty Invigilation Conflict (Binary):** A faculty member (invigilator) cannot proctor two different examinations during the exact same time slot.

5. **Cognitive Fatigue Rest Constraint (Higher-Order Temporal):** To prevent student burnout, courses belonging to the exact same batch cannot be scheduled in both the AM and PM slots of the *same day*.

What you need to do

The current boilerplate uses a naive sequential variable selection method. If left unaltered, the solver will experience a combinatorial explosion and fail to resolve complex schedules within the time limit. You must rewrite the mrv_select_variable function to implement the **Minimum Remaining Values (MRV)** heuristic (the fail-first heuristic). The solver must dynamically select the next unassigned course that possesses the fewest remaining legal (Time_Slot, Room) combinations in its domain based on the current state of assignments.

Boilerplate Code + Testing

My provided code contains the structural skeleton of the CSP class, an incomplete naive solver algorithm, a pre-built data dictionary representing the problem space, and an automated test runner. You are explicitly prohibited from altering the test runner suite or the class initialization mechanics. You must construct the required logic exclusively within the is_consistent, mrv_select_variable, and backtrack methods.

The Code can be found @this link

Test Cases

The first test case is already included in the boilerplate code. Make sure to check with the 2nd test case too which will actually check your code strength. Also, during the code explanation part of the test, you will have to tell me the number of approximate operations your code performed for the 2nd test case and why so.

The test samples part to be modified	The Excepted Output

`# Define variables`
`courses = ['CSE111', 'CSE112', 'CSE221', 'CSE222', 'CSE331', 'CSE332']`
 	
`# Define time slots: Day 1 & Day 2 (AM and PM)`
`time_slots = [(1, 'AM'), (1, 'PM'), (2, 'AM'), (2, 'PM')]`
 	
`# Define rooms`
`rooms = ['Room_101', 'Room_102', 'Lab_401']`
 	
`# Create absolute domain: Cartesian product of slots and rooms`
`domain = [(ts, r) for ts in time_slots for r in rooms]`
 	
`# Define Course Metadata`
`course_meta = {`
	`Initializing Exam Scheduling CSP...`
	`Executing Backtracking Search...`
	`SUCCESS: All constraints met! Valid Timetable Generated:`
	`{`
	`"CSE111": [`
	`[`
	`1,`
	`"AM"`
	`],`
	`"Lab_401"`
	`],`
	`"CSE222": [`
	`[`
	`1,`

```
        'CSE111': {'batch': 'Batch_1', 'invigilator': 'Prof_Rahman', 'lab_required':
True},
        'CSE112': {'batch': 'Batch_1', 'invigilator': 'Dr_Karim',   'lab_required':
False},
        'CSE221': {'batch': 'Batch_2', 'invigilator': 'Prof_Rahman', 'lab_required':
False},
        'CSE222': {'batch': 'Batch_2', 'invigilator': 'Dr_Hassan',  'lab_required':
True},
        'CSE331': {'batch': 'Batch_3', 'invigilator': 'Dr_Karim',   'lab_required':
False},
        'CSE332': {'batch': 'Batch_3', 'invigilator': 'Dr_Nahar',   'lab_required':
False},
    }
```

```
            "PM"
        ],
        "Lab_401"
    ],
    "CSE112": [
        [
            2,
            "AM"
        ],
        "Room_101"
    ],
    "CSE221": [
        [
            2,
            "AM"
        ],
        "Room_102"
    ],
    "CSE331": [
        [
            1,
            "AM"
        ],
        "Room_101"
    ],
    "CSE332": [
        [
            2,
            "PM"
        ],
        "Room_101"
    ]
}
Nodes expanded (Backtracks): 0
```

```
courses = ['ENG101', 'ENG102', 'PHY101', 'PHY102', 'CS101', 'CS102', 'CS103',
'MTH101']
time_slots = [(1, 'AM'), (1, 'PM'), (2, 'AM'), (2, 'PM'), (3, 'AM')]
rooms = ['Hall_A', 'Hall_B', 'Comp_Lab_1']

course_meta = {
    'ENG101': {'batch': 'B1', 'invigilator': 'Prof_X', 'lab_required': False},
    'ENG102': {'batch': 'B2', 'invigilator': 'Prof_Y', 'lab_required': False},
    'PHY101': {'batch': 'B1', 'invigilator': 'Prof_Z', 'lab_required': True},
    'PHY102': {'batch': 'B2', 'invigilator': 'Prof_X', 'lab_required': True},
    'CS101':  {'batch': 'B3', 'invigilator': 'Prof_Y', 'lab_required': True},
    'CS102':  {'batch': 'B3', 'invigilator': 'Prof_Z', 'lab_required': False},
    'CS103':  {'batch': 'B4', 'invigilator': 'Prof_X', 'lab_required': True},
    'MTH101': {'batch': 'B4', 'invigilator': 'Prof_Y', 'lab_required': False}
}
```

```
Initializing Exam Scheduling CSP...
Executing Backtracking Search...
SUCCESS: All constraints met! Valid Timetable Generated:
{
    "PHY101": [
        [
            1,
            "AM"
        ],
        "Comp_Lab_1"
    ],
    "PHY102": [
        [
            1,
            "PM"
        ],
        "Comp_Lab_1"
    ],
    "CS101": [
        [
            2,
            "AM"
        ],
        "Comp_Lab_1"
    ],
    "CS103": [
        [
            2,
            "PM"
        ],
        "Comp_Lab_1"
    ],
    "ENG101": [
        [
            2,
            "AM"
        ],
        "Hall_A"
    ],
    "ENG102": [
        [
            2,
            "PM"
        ],
        "Hall_A"
    ],
    "CS102": [
        [
            1,
            "PM"
        ],
        "Hall_A"
    ],
    "MTH101": [
        [
            1,
            "AM"
        ],
        "Hall_A"
    ]
}
Nodes expanded (Backtracks): 0
```

HAPPY CODING

